



Alumnos—

Luis Francisco Salido Varela

Mario Alejandro Preciado Guerrero

Julio Cesar Rojas

Manuel Donato Hernandez Burgos

José Antonio González Valle

ID's—

00000187523

00000252940

00000260308

00000181539

00000235621

Proyecto—

Mesa Lista

Fecha—

15 de mayo de 2026

Materia—

Diseño de software

Profesor—

Felix Daniel Campoa Toledo

2.1 Descripción de la Problemática	4
2.2 Solución Propuesta	4
2.3 Diagramas Requeridos	5
2.3.1 Diagrama de Casos de Uso	6
2.3.2 Diagramas de Secuencia	7
2.3.3 Diagrama de Clases	8
2.3.4 Diagrama de Subsistemas	9
2.4 Explicación de Decisiones de Diseño	10
2.4.1 Selección de Patrones de Diseño	10
2.4.2 Justificación de la Arquitectura Elegida	10
2.4.3 Consideraciones de Escalabilidad, Mantenibilidad y Extensibilidad	10
2.5 Patrones de Diseño Implementados	11
2.5.1 Builder	11
3. Conclusión	12
4. Anexos	

2.1 Descripción de la Problemática

Actualmente, muchos restaurantes de comida casual, cafeterías y establecimientos con servicio presencial administran sus reservaciones y disponibilidad de mesas de forma manual o mediante procesos poco centralizados. Esto provoca dificultades para controlar la ocupación en tiempo real, errores al momento de asignar espacios, tiempos de espera innecesarios y problemas en la organización del servicio durante horarios de alta demanda.

En la mayoría de los casos, los clientes deben comunicarse por llamada telefónica, mensajes o acudir físicamente al establecimiento para consultar disponibilidad y realizar una reservación. Además, algunos restaurantes no cuentan con mecanismos digitales que permitan confirmar reservaciones mediante pagos anticipados, ocasionando cancelaciones de último momento, mesas apartadas que no son utilizadas y pérdidas económicas para el negocio.

El proyecto “MesaLista” surge como una solución orientada a digitalizar y optimizar el proceso de reservaciones en restaurantes, permitiendo a los usuarios consultar restaurantes disponibles, visualizar información relevante del establecimiento, seleccionar preferencias de reservación y realizar pagos digitales por reservación desde una plataforma centralizada.

Los principales usuarios involucrados dentro del sistema son los clientes y los administradores de restaurantes. Los clientes requieren una plataforma sencilla que les permita registrarse, iniciar sesión, consultar restaurantes, verificar disponibilidad, realizar reservaciones y consultar su historial de reservas. Por otro lado, los administradores necesitan herramientas para registrar restaurantes, administrar áreas disponibles, gestionar menús informativos y controlar la disponibilidad de reservaciones dentro del sistema.

Entre las necesidades detectadas se encuentra la automatización del proceso de reservación, la reducción de errores humanos en la asignación de espacios, la mejora en la experiencia del cliente y la integración de métodos de pago digitales que permitan confirmar reservaciones de manera segura y eficiente. También se identificó la necesidad de contar con información centralizada que facilite la administración de restaurantes y permita mantener un mejor control operativo.

El entorno presenta distintas limitaciones y condiciones que deben considerarse durante el desarrollo del sistema. Una de las principales es la dependencia de conexión a internet para consultar disponibilidad y procesar pagos digitales mediante APIs externas. Asimismo, el sistema debe ser intuitivo y fácil de utilizar para distintos tipos de usuarios, considerando que algunos restaurantes pequeños no cuentan con infraestructura tecnológica avanzada. Además, el sistema debe manejar correctamente la concurrencia de reservaciones para evitar conflictos de disponibilidad y garantizar una experiencia confiable tanto para clientes como para administradores.

2.2 Solución Propuesta

La solución propuesta por el equipo consiste en el desarrollo de “MesaLista”, un sistema digital orientado a la gestión de reservaciones para restaurantes, diseñado para facilitar la interacción entre clientes y establecimientos mediante una plataforma centralizada y accesible. El sistema busca optimizar el proceso de reservación, mejorar la organización operativa de los restaurantes y brindar una experiencia más eficiente y cómoda para los usuarios.

El objetivo principal del sistema es permitir que los clientes puedan consultar restaurantes, verificar disponibilidad, realizar reservaciones y confirmar su asistencia mediante pagos digitales, mientras que los administradores de restaurantes podrán gestionar información relacionada con sus establecimientos, áreas disponibles y disponibilidad de reservaciones.

Entre las funcionalidades clave del sistema se encuentran el registro e inicio de sesión de usuarios, la consulta de restaurantes registrados, la visualización de información relevante del establecimiento, la consulta de disponibilidad, la realización de reservaciones y el pago digital por reservación mediante integración con una pasarela de pago externa. Además, el sistema contempla funcionalidades administrativas para la gestión de restaurantes, administración de áreas y consulta de historial de reservaciones.

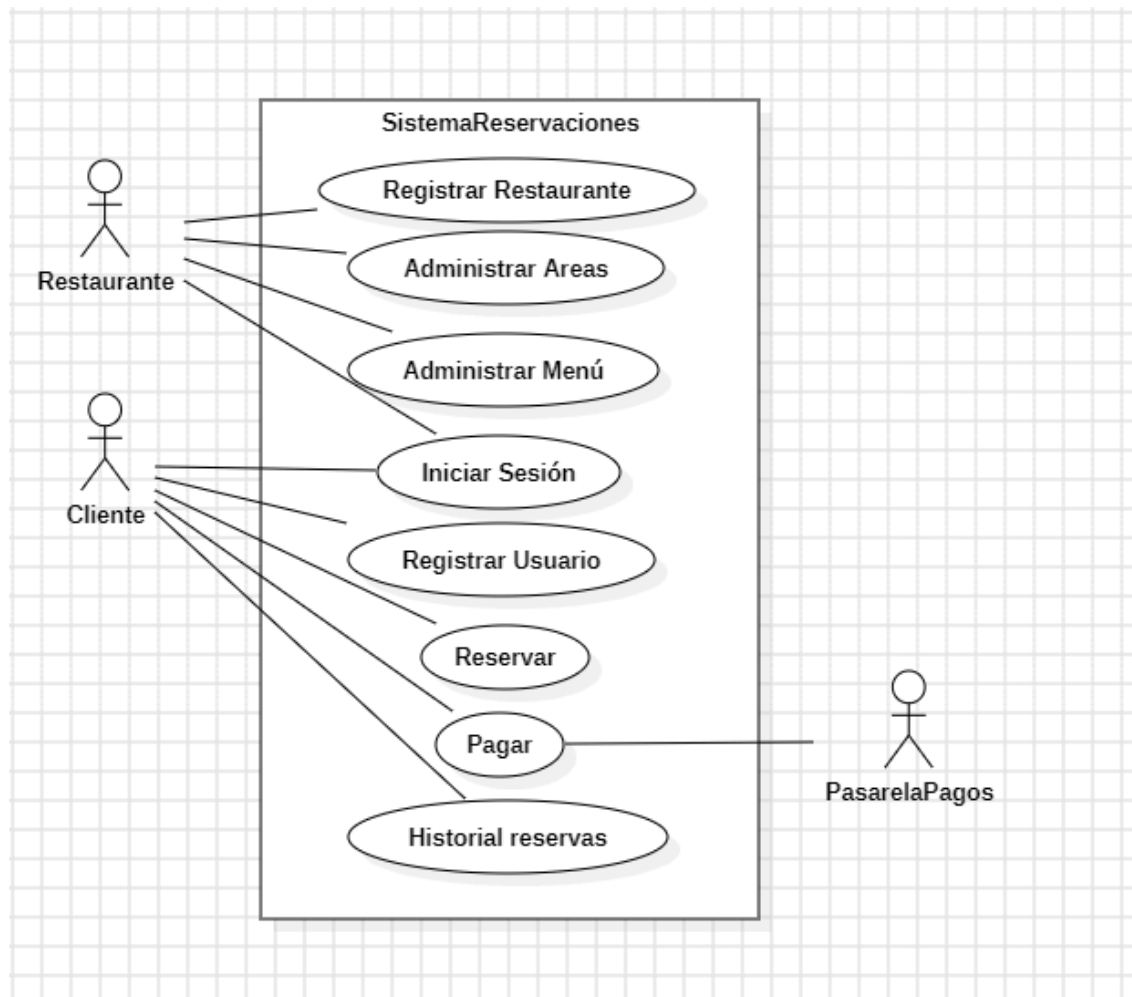
Se espera que la implementación de MesaLista aporte distintos beneficios tanto para los clientes como para los restaurantes. Para los usuarios, permitirá reducir tiempos de espera, facilitar el acceso a reservaciones y mejorar la experiencia de uso mediante una plataforma digital centralizada. Para los restaurantes, permitirá disminuir errores en la administración de reservaciones, reducir cancelaciones mediante pagos anticipados y mejorar el control operativo y administrativo del negocio.

El alcance del proyecto contempla el desarrollo de un sistema funcional enfocado principalmente en la gestión de reservaciones y pagos asociados a las mismas, dejando fuera funcionalidades más complejas relacionadas con pedidos de alimentos, logística interna de cocina o administración avanzada de inventarios. Asimismo, el sistema depende de conexión a internet para operar correctamente y requiere integración con servicios externos para el procesamiento de pagos digitales. Como parte de las restricciones, el proyecto se desarrollará bajo una arquitectura modular utilizando Java y tecnologías relacionadas, considerando el tiempo disponible del equipo y los recursos tecnológicos accesibles durante el desarrollo.

2.3 Diagramas Requeridos

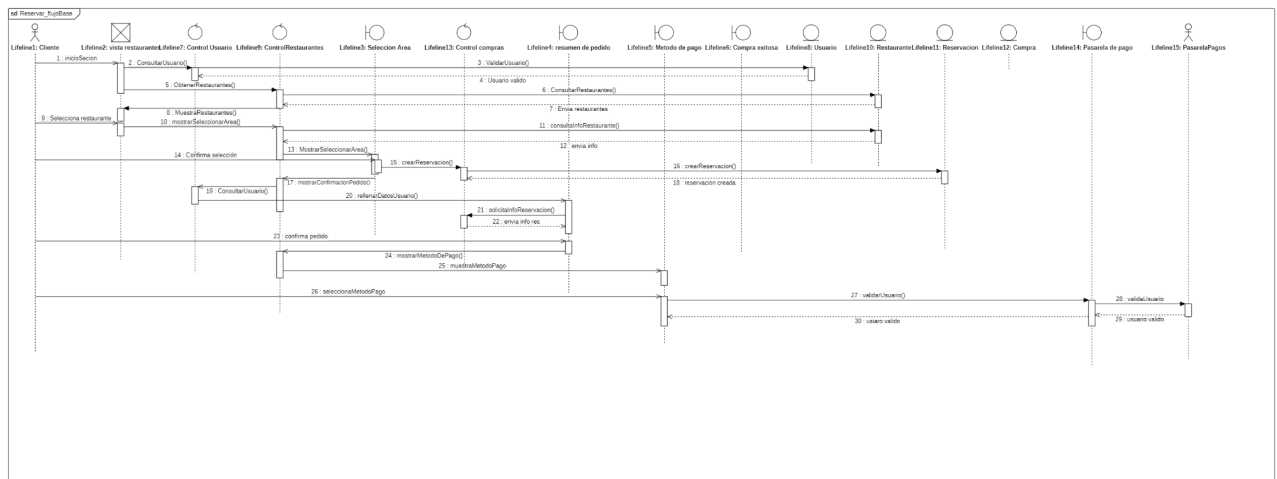
Se anexaron los diagramas solicitados:

2.3.1 Diagrama de Casos de Uso



El diagrama de casos de uso representa las principales interacciones entre los actores del sistema y las funcionalidades generales de la plataforma MesaLista. Los actores principales son el Cliente, el Administrador del Restaurante y la Pasarela de Pagos como sistema externo. A través de estos casos de uso se modelan las operaciones relacionadas con el registro de usuarios, reservaciones, pagos digitales y administración de restaurantes dentro del sistema.

Asimismo, los diagramas permiten visualizar los mensajes intercambiados entre los distintos componentes del sistema para ejecutar correctamente el flujo de reservación y confirmación de pago.

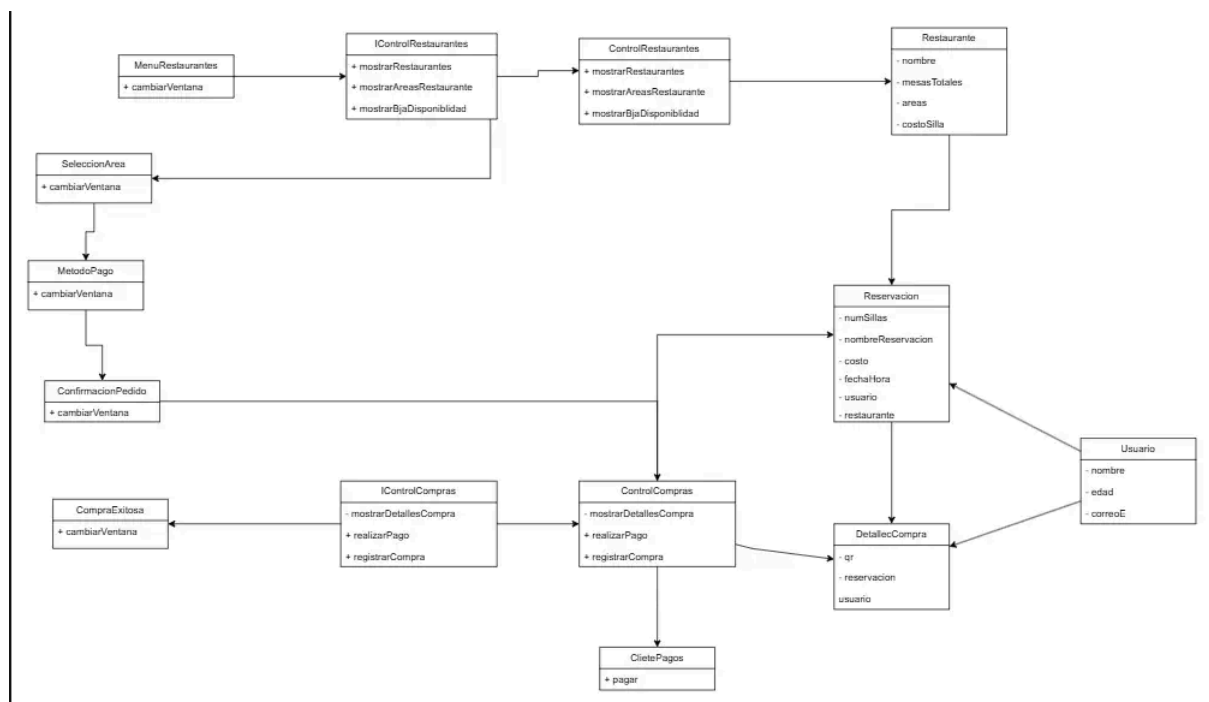


2.3.3 Diagrama de Clases

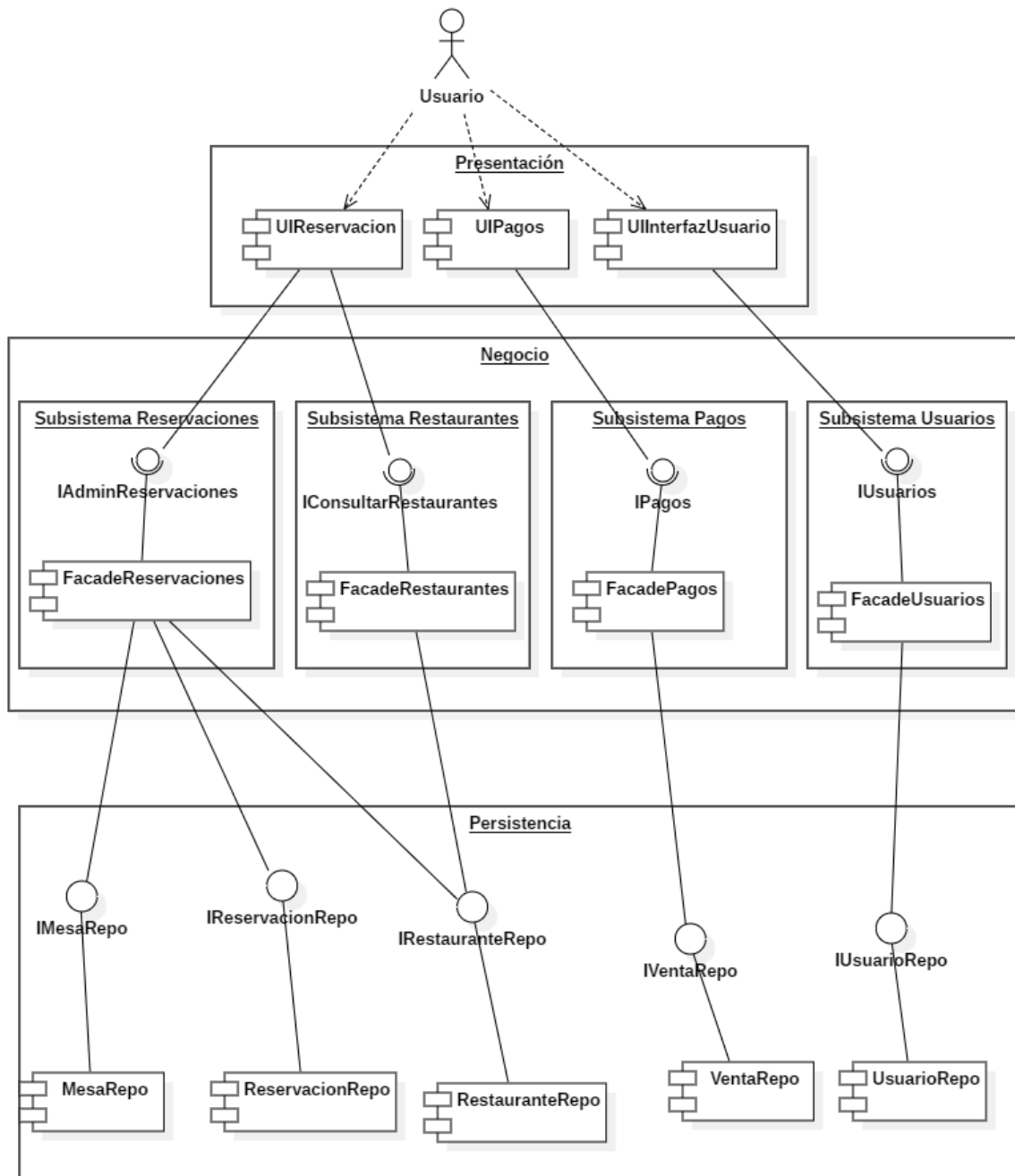
El diagrama de clases representa la estructura principal del sistema “MesaLista”, mostrando las clases, atributos, métodos y relaciones más importantes. Para el desarrollo se utilizó una arquitectura basada en Boundary, Control y Entity, separando la interfaz de usuario, la lógica de negocio y las entidades del sistema.

Entre las clases principales se encuentran Usuario, Restaurante, Reservacion y DetalleCompra, encargadas de almacenar la información relacionada con clientes, restaurantes, reservaciones y pagos. Asimismo, las clases de control gestionan las operaciones del sistema, como consultar disponibilidad, realizar reservaciones y procesar pagos.

El diagrama también muestra las relaciones entre las clases y la interacción existente entre los distintos componentes del sistema durante el flujo de reservación.



2.3.4 Diagrama de Subsistemas



El diagrama de subsistemas representa la arquitectura general del sistema "MesaLista", mostrando la separación de responsabilidades entre las distintas capas del proyecto. La arquitectura se encuentra dividida en tres niveles principales: Presentación, Negocio y Persistencia.

La capa de Presentación contiene las interfaces gráficas encargadas de la interacción con el usuario, permitiendo realizar operaciones relacionadas con reservaciones, pagos y administración de usuarios. La capa de Negocio integra los subsistemas responsables de procesar la lógica del sistema, incluyendo reservaciones, restaurantes, pagos y usuarios mediante el uso de fachadas e interfaces que facilitan la comunicación entre módulos.

Finalmente, la capa de Persistencia se encarga del acceso y almacenamiento de información mediante repositorios especializados para cada entidad principal del sistema, permitiendo mantener organizada la administración de datos relacionados con reservaciones, restaurantes, usuarios y ventas.

Esta separación modular facilita el mantenimiento, reutilización y escalabilidad del sistema, permitiendo una mejor organización del proyecto y reduciendo el acoplamiento entre componentes.

2.4 Explicación de Decisiones de Diseño

2.4.1 Selección de Patrones de Diseño

El equipo seleccionó los patrones de diseño basándose en los problemas concretos que presentaba el sistema. Se eligió el patrón Singleton para garantizar que existiera una única conexión activa a la base de datos en toda la aplicación, evitando el desperdicio de recursos. El patrón Facade fue elegido para cada subsistema porque el controlador de presentación necesitaba operar con módulos complejos (reservaciones, pagos, usuarios) sin conocer su implementación interna. El patrón Adapter fue necesario al integrar una API externa (PayPal) cuya interfaz no coincidía con el contrato definido internamente. El patrón Builder se aplicó en Reservación porque este objeto tiene muchos atributos opcionales y su construcción paso a paso mejora la legibilidad.

2.4.2 Justificación de la Arquitectura Elegida

Se adoptó una arquitectura por capas modular, donde el proyecto está dividido en submódulos Maven independientes (MesaListaPersistencia, MesaListaDAOs, MesaListaDTO, MesaListaMappers, SubSistema*, MesaListaPresentacion). Esta decisión permite que cada capa tenga una responsabilidad única y que los subsistemas puedan desarrollarse y mantenerse de forma separada. La elección de MongoDB como base de datos responde a la naturaleza del dominio: los restaurantes tienen estructuras variables (áreas, menús, platillos anidados), lo que encaja mejor en un modelo de documentos que en un esquema relacional rígido. La capa de presentación nunca accede directamente a los repositorios ni a los DAOs; siempre pasa por ControlOperaciones, que actúa como punto de entrada único al sistema. Esto centraliza la lógica de negocio (por ejemplo, cobrar antes de guardar la reservación) en un solo lugar.

2.4.3 Consideraciones de Escalabilidad, Mantenibilidad y Extensibilidad

- Escalabilidad: Al usar MongoDB, agregar nuevos campos a documentos existentes (ej. añadir preferencias a un usuario) no requiere migrar el esquema de toda la base de datos. El uso de interfaces en las fachadas (IUsuarioFacade, IReservacionesFacade, etc.) permite sustituir cualquier subsistema por una implementación distribuida o basada en microservicios sin tocar el controlador.
- Mantenibilidad: La separación en capas (Entidad- DAO- Mapper- Repository- Facade- ControlOperaciones) hace que un cambio en la forma en que MongoDB almacena un campo solo impacte al Mapper correspondiente, sin propagar el cambio a otras capas.

- Extensibilidad: La interfaz IPasarelaPago permite agregar en el futuro un StripeAdapter o MercadoPagoAdapter sin modificar PagosFacade ni ninguna otra clase, solo creando un nuevo adaptador que implemente la misma interfaz.

2.5 Patrones de Diseño Implementados

2.5.1 Builder

- Problema que resuelve: Construir objetos complejos con muchos atributos de forma legible y controlada, evitando constructores con largas listas de parámetros que son difíciles de leer y propensos a errores.
- Razón por la que fue elegido: La entidad Reservacion tiene 9 atributos. Un constructor con 9 parámetros es difícil de usar correctamente (es fácil confundir el orden). El Builder permite construir el objeto paso a paso, especificando solo los campos necesarios con nombres explícitos.
- Cómo se implementa: La clase ReservacionBuilder tiene un método por cada atributo que retorna this (encadenamiento fluido), y un método build() que construye el objeto final. La entidad Reservacion expone el Builder mediante un método estático:

```
// Uso del Builder
Reservacion r = Reservacion.builder()
    .numPersonas(4)
    .folio("RSV-001")
    .fechaHora(LocalDateTime.now())
    .restauranteId(idRestaurante)
    .build();
```

2.5.2 DAO (Data Access Object)

- Problema que resuelve: Separar la lógica de acceso a la base de datos del resto de la aplicación, centralizando todas las consultas y operaciones sobre MongoDB en clases especializadas.
- Razón por la que fue elegido: Sin este patrón, las consultas a MongoDB estarían dispersas por toda la aplicación. Con el DAO, si se cambia la base de datos o la forma de consultar, solo se modifica la clase DAO correspondiente.
- Cómo se implementa: La interfaz genérica IDAO<T, K> define el contrato CRUD (insertar, buscarPorId, actualizar, eliminar). Cada entidad tiene su implementación concreta (ReservacionDAO, UsuarioDAO, RestauranteDAO, VentaDAO) que trabaja directamente con colecciones de MongoDB.

2.5.3 DTO + Mapper

- Problema que resuelve: Transferir datos entre capas sin exponer las entidades internas ni los documentos de la base de datos. Los Mappers se encargan de convertir entre el formato de MongoDB (Document BSON) y los objetos que circulan entre capas (DTO).
- Razón por la que fue elegido: Exponer las entidades de persistencia directamente en la capa de presentación crearía un acoplamiento que impediría modificar el modelo de datos sin impactar las pantallas. Los DTOs son objetos planos y portables que no dependen de ningún framework.
- Cómo se implementa: La interfaz genérica IMapper<D> define toDocument(D dto) y toDto(Document doc). Cada dominio tiene su mapper concreto (ReservacionMapper, UsuarioMapper, etc.). El Repository de cada subsistema orquesta la conversión: recibe un DTO, pide al Mapper que lo convierta a Document, y se lo pasa al DAO para persistirlo.

```
// ReservacionRepository.java
public void guardar(ReservacionDTO dto) {
    Document doc = mapper.toDocument(dto); // DTO → BSON
    dao.insertar(doc);
}

public ReservacionDTO buscarPorId(String id) {
    return mapper.toDto(dao.buscarPorId(new ObjectId(id)));
}
// BSON → DTO
```

3. Conclusión

3.1 Evaluación de la eficiencia de la solución desarrollada.

La solución desarrollada permitió mejorar el proceso de reservaciones mediante la digitalización de actividades que anteriormente se realizaban de forma manual. El sistema facilita la consulta de restaurantes, disponibilidad y realización de reservaciones desde una sola plataforma, reduciendo tiempos de espera y mejorando la organización de la información.

Asimismo, la integración del pago digital por reservación permite confirmar reservaciones de manera más eficiente, disminuyendo cancelaciones y mejorando el control administrativo de los restaurantes. La arquitectura modular implementada también favorece la organización del sistema, permitiendo separar la lógica de

negocio, las interfaces y las entidades para facilitar el mantenimiento y escalabilidad del proyecto.

Durante el desarrollo se logró una comunicación adecuada entre los distintos módulos del sistema, permitiendo que las funcionalidades principales operaran correctamente dentro del flujo de reservación y pago.

3.2 Reflexión sobre el proceso de diseño de software aplicado.

El proceso de diseño precedió correctamente a la implementación. Antes de escribir código funcional, se tomaron decisiones estructurales clave: qué patrones de diseño aplicar, cómo separar las responsabilidades entre capas y cómo definir los contratos entre subsistemas mediante interfaces (`IUsuarioFacade`, `IPasarelaPago`, `IDAO<T,K>`, `IMapper<D>`). Esta disciplina se refleja en la coherencia del código resultante: cada clase tiene una responsabilidad bien delimitada y los cambios en una capa no se propagan innecesariamente a las demás. Sin embargo, el proceso también evidenció que no siempre existe una correspondencia perfecta entre el diseño planificado y la implementación final; el patrón Builder, aunque correctamente implementado en la capa de persistencia, no llegó a integrarse en el flujo real del sistema, lo cual representa una oportunidad de mejora identificada durante el desarrollo.

3.3 En qué medida el diseño contribuyó al correcto desarrollo del sistema.

El diseño fue determinante para el desarrollo ordenado del sistema. La definición temprana de interfaces como `IPasarelaPago` permitió que el subsistema de pagos se desarrollara de forma independiente al resto, ya que los demás módulos solo necesitaban conocer el contrato, no la implementación concreta. De igual forma, la existencia de `ControlOperaciones` como punto de entrada único a todos los subsistemas evitó que la capa de presentación acumulara lógica de negocio, manteniendo las pantallas enfocadas exclusivamente en la interacción con el usuario. El uso del patrón Singleton en la conexión a la base de datos garantizó que en ningún momento existieran múltiples conexiones abiertas simultáneamente, previniendo problemas de recursos desde el diseño y no como una corrección posterior.

3.4 Aprendizajes obtenidos durante el proyecto.

Este proyecto dejó aprendizajes concretos en varias dimensiones. En el aspecto técnico, se comprendió en la práctica la diferencia entre diseñar una arquitectura y ejecutarla: definir interfaces, separar capas y aplicar patrones no es un trámite formal sino una decisión que impacta directamente en la facilidad con la que se

puede modificar o extender el sistema. Se aprendió también que la elección de la base de datos debe responder a la naturaleza del dominio y no a la familiaridad del equipo con una tecnología específica. En el aspecto del proceso, quedó claro que un patrón de diseño solo aporta valor cuando se implementa completamente y se integra en el flujo real del sistema; de lo contrario, añade complejidad sin beneficio. Finalmente, la integración con una API externa como PayPal evidenció la importancia del patrón Adapter: aislar las dependencias externas mediante interfaces propias protege al sistema de cambios en bibliotecas de terceros y facilita la sustitución de proveedores sin necesidad de modificar la lógica central de la aplicación.

4. Anexos

Se agregan las ligas para visualizar y descargar el proyecto desarrollado dentro de un repositorio de GitHub, así como el enlace de Google Drive donde se encuentran los diagramas solicitados.

Github:

<https://github.com/MarioAlejandroPreciadoGuerrero/MesaLista>

Drive:

https://drive.google.com/drive/folders/1nU8_NfPT5qNTAdImhhdt-xBSwf7Dv0ts